

BSc (Hons) in **Computer Science**

Faculty of Computing

SE3062 : Intelligent Systems

Year 3 · Semester 1 · 2026

Practical 07

> **Module Objective:** In previous weeks, your Drow Ranger agent used A* Search to find a physical path to the goal. However, it blindly assumed every non-wall tile was safe. Today, we implement a **Knowledge Base (KB)** and a **Forward Chaining Inference Engine**. Your agent must now mathematically prove a cell is logically *Feasible* before taking a step, using real-time rules about map vision and enemy threats.

> **Before you start:** This lab continues the project from Labs 1–4. Open the same project and check that `SearchAgent.astar_search()` in `agent.py` works.
>
> - Parts 1, 2 & 4: new files (`logic_engine.py` , `test_logic.py`) in the project folder.
> - Part 3: edit your existing `agent.py` .
> - **Create a separate branch `lab-07`** and start your work there. When finished, **commit your code and push the branch**, as you did in Labs 1–4.
> - **Upload this worksheet's answered PDF to the submission portal.**

Part 1: The Logic Engine Architecture

Step 1.1: Building the Knowledge Base (KB)

> Instead of hardcoding hundreds of nested `if-else` statements in our agent's movement logic, we will build a declarative Knowledge Base. You need to design a Python class that stores **Facts** (current percepts) and **Rules** (Horn Clauses).

1. Create a new file named `logic_engine.py` .

2. Implement a `KnowledgeBase` class based on the following structural requirements:

```
CLASS KnowledgeBase:
    // Attributes
    DECLARE facts AS Set                    // To store uniqu
    DECLARE rules AS List                   // To store rules

    // Methods
    FUNCTION tell fact(fact string):
        ADD fact_string TO facts

    FUNCTION tell rule(premise list, conclusion string):
        APPEND Tuple(premise_list, conclusion_string) TO r

    FUNCTION clear facts():
        EMPTY the facts Set
```

Part 2: Implementing Forward Chaining

Step 2.1: The Inference Algorithm

> The agent needs an Inference Engine to deduce new information from its raw sensors. Translate the following Data-Driven **Forward Chaining** algorithm into a Python method inside your `KnowledgeBase` class.

1. Write the `forward_chain()` method.
2. Ensure you use a `while` loop that continues to iterate over the rules until a full pass results in absolutely no new facts being deduced.

```
FUNCTION forward chain():
    DECLARE new_facts_added = TRUE

    WHILE new_facts_added == TRUE:
```

```
        new_facts_added = FALSE

        FOR EACH (premises, conclusion) IN rules:
            IF conclusion IS NOT IN facts:

                // Modus Ponens Check
                IF ALL premises ARE IN facts:
                    ADD conclusion TO facts
                    new_facts_added = TRUE
```

Part 3: Hooking Logic into the Grid Game

Step 3.1: Defining the Game Constraints

> Now we translate the game's safety constraints into our logic engine using Propositional Logic.

1. In `agent.py`, add `from logic_engine import KnowledgeBase`, then create the KB in `SearchAgent.__init__`: `self.kb = KnowledgeBase()`.
2. Use your `tell_rule` method to define the following safety rules:
   Rule 1: `TargetVisible ∧ HasDust ⇒ SafeToEngage`
   Rule 2: `SafeToEngage ∧ BloodseekerMissing ⇒ Retreat`

The game does not produce these facts, so you supply them per tile as a dictionary: `{(x, y): [facts]}`. A tile where `Retreat` is deduced is unsafe to enter.

Step 3.2: Validating Feasibility in A*

> Modify your existing A* pathfinding algorithm to consult the Knowledge Base before expanding a node.

1. Add a parameter `tile_facts=None` to `astar_search()` (treat `None` as `{}`).

2. Locate the neighbor-generation loop in your A* code (where you normally check for physical walls).
3. Before adding a neighbor to the `open_list`, clear the KB facts.
4. Feed that tile's facts into the KB: `for fact in tile_facts.get(successor, []): self.kb.tell_fact(fact)`.
5. Run `self.kb.forward_chain()`.
6. If `'Retreat'` is deduced and exists in `self.kb.facts`, mark the tile as **Infeasible**. Skip it (do not add it to the open list), even if it is physically reachable.

Step 3.3: Run It in the Grid

1. In `sense_and_act()`, define sample facts and pass them to A*:

```
tile_facts = {
    (1, 0): ['TargetVisible', 'HasDust', 'BloodseekerMissi
    (0, 1): ['TargetVisible', 'HasDust'],
}
self.plan = self.astar_search(position, nearest_food, walls
```

2. Run `visual_grid_game.py`. The agent should never step onto `(1, 0)`.

Part 4: Self-Evaluation Test Cases

4.1 Automated Validation

Create a file named `test_logic.py`. Copy and run the following Python assert statements to verify your Forward Chaining engine works correctly mathematically before you attempt to integrate it into the visual grid.

```
 from logic_engine import KnowledgeBase

def test_forward_chaining():
    kb = KnowledgeBase()

    # Add Domain Rules
    kb.tell_rule(['TargetVisible', 'HasDust'], 'SafeToEnga
    kb.tell_rule(['SafeToEngage', 'BloodseekerMissing'], '

    # Test Case 1: Safe Engagement
    kb.clear_facts()
    kb.tell_fact('TargetVisible')
    kb.tell_fact('HasDust')
    kb.forward_chain()
    assert 'SafeToEngage' in kb.facts, "Test 1 Failed: Sho
    assert 'Retreat' not in kb.facts, "Test 1 Failed: Shou

    # Test Case 2: Unsafe Engagement (Bloodseeker Missing)
    kb.clear_facts()
    kb.tell_fact('TargetVisible')
    kb.tell_fact('HasDust')
    kb.tell_fact('BloodseekerMissing')
    kb.forward_chain()
    assert 'Retreat' in kb.facts, "Test 2 Failed: Should d

    print("✅ All Logic Engine Test Cases Passed!")

if __name__ == "__main__":
    test_forward_chaining()
```

Part 5: Theory-to-Implementation Mapping

> Based on the code you just wrote and the lecture on Logical Reasoning, complete the following analytical questions to explain *how* your code implements the theory.

1. Reachability vs. Feasibility (Apply)
In Step 3.2, you modified the A* algorithm's neighbor-generation loop. How does your new code differentiate between checking if a node is "Reachable" (like a wall collision) versus checking if it is "Feasible"? Explain how the Knowledge Base enforces feasibility.

Reachability is a physical question: can the agent actually move onto a tile? In my A* code, the `neighbours` method checks it by rejecting tiles that are outside the grid or are walls, so those tiles are never even generated as successors. Feasibility is a logical question: even if the tile is physically reachable, is it safe to enter given what the agent knows? The code checks this separately, after a tile has passed the reachability check and before it is added to the open list. It clears the KB, tells it the facts for that tile from `tile_facts`, and runs `forward_chain()`. If `'Retreat'` appears in `kb.facts`, the tile is skipped with `continue`, so it never enters the open list. The Knowledge Base enforces feasibility because the rules decide what can be deduced. The tile (1, 0) has the facts TargetVisible, HasDust and BloodseekerMissing. Rule 1 gives SafeToEngage, and Rule 2 then gives Retreat, so the tile is marked infeasible. A* then finds a route around it, even though a wall does not block it.

2. Declarative vs. Procedural Paradigms (Analyze)
Why did we build a `KnowledgeBase` class with a `forward_chain()` loop instead of simply writing `if 'TargetVisible' in percepts and 'HasDust' in percepts:` directly inside the agent's movement code? What happens to the agent's code complexity if we add 50 new game rules?

In a declarative design, we describe what is true (the facts and the rules) and the inference engine works out the consequences. The agent's movement code stays the same no matter how many rules there are, because `forward_chain()` is a general loop that works on any rules. Writing `if 'TargetVisible' in percepts and 'HasDust' in percepts:` inside the movement code is procedural. It mixes the game knowledge with the control code, and chained conclusions, such as SafeToEngage leading to Retreat, would need nested ifs in exactly the right order. With 50 new rules, the procedural code would grow into a long tree of nested if-else statements. It would be hard to read, easy to break, and

each new rule could interact with the old ones in ways that are hard to see. With the Knowledge Base, adding 50 rules means adding 50 `tell_rule` calls. The agent code does not change, and the rules can be read, tested and changed on their own, as `test_logic.py` does.

3. Modus Ponens & Horn Clauses (Understand)
The rule `['TargetVisible', 'HasDust'] ⇒ 'SafeToEngage'` is formally known as a Horn Clause. Briefly explain how your Python implementation of the `forward_chain()` `while` loop acts as a mathematical execution of the *Modus Ponens* inference rule.

A Horn clause has the form P1 ∧ P2 ∧ ... ∧ Pn ⇒ Q, which is at most one positive conclusion with a conjunction of premises, for example TargetVisible ∧ HasDust ⇒ SafeToEngage. Modus Ponens says that if P ⇒ Q is true and P is true, then Q must be true. In `forward_chain()` , each `(premises, conclusion)` tuple is a Horn clause. The check `all(p in self.facts for p in premises)` tests whether every premise is already known to be true, and if it is, the line `self.facts.add(conclusion)` adds the conclusion, which is the Modus Ponens step. The `while` loop repeats this over all rules until a full pass adds no new fact. This matters because one deduction can make another rule's premises true. Here, SafeToEngage is deduced first, and then, together with BloodseekerMissing, it gives Retreat. When the loop stops, the fact set is a fixed point, which contains everything that can be derived from the rules and the starting facts, so the algorithm is sound (it only derives true conclusions) and complete for Horn clauses.

Finalize and Lock Lab 07 Implementation

FACULTY OF COMPUTING